Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.03.01 (ИНФОРМАТИКА И ВЫЧИСЛИТЕЛЬНАЯ ТЕХНИКА)**

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовой работе

Название: База данных «Онлайн-кинотеатр»

Дисциплина: Базы данных

Студент	ИУ6-45Б		В.А.Ивличева
	(Группа)	(Подпись, дата)	(И.О. Фамилия)
Руководитель			М.А.Скворцова
		(Подпись, дата)	(И.О. Фамилия)

Москва, 2025

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой ___ИУ6__
(Индекс)
_____ А.В. Пролетарский
(И.О. Фамилия)
« ___ » _____ 2025 г.

З А Д А Н И Е
на выполнение курсовой работы

по дисциплине ____Базы данных_____

Студент группы ____ИУ6-45_____

_____Ивличева Валерия Андреевна_____
(Фамилия, имя, отчество)

Тема курсовой работы ____Онлайн-кинотеатр_____
_____
_____

Направленность КР (учебная, исследовательская, практическая, производственная, др.)
_учебная_____

Источник тематики (кафедра, предприятие, НИР) ____кафедра_____

График выполнения КР: 25% к 4 нед., 50% к 7 нед., 75% к 11 нед., 100% к 14 нед.

Техническое задание

Необходимо разработать базу данных «Онлайн-кинотеатр», содержащую не менее 7 связных таблиц. Основная сущность должна содержать не менее 1 млн. записей, остальные не менее 100 записей. Разработать инфологическую и даталогическую модель базы данных. В базе данных должно быть разработано не менее 7 сложных/вложенных запросов. В одном из запросов реализовать возможность его формирования по условию преподавателя.

Оформление курсовой работы:
Расчетно-пояснительная записка (РПЗ) на не менее 25 листах формата А4.

Дата выдачи задания «07» февраля 2025 г.

Руководитель курсовой работы _____ М.А.Скворцова
(Подпись, дата) (И.О. Фамилия)

Студент _____ В.А.Ивличева
(Подпись, дата) (И.О. Фамилия)

Примечание: Задание оформляется в двух экземплярах: один выдается студенту, второй хранится на кафедре.

РЕФЕРАТ

Расчётно-пояснительная записка по курсовой работе на тему «Онлайн-кинотеатр» состоит из 37 страниц, 18 рисунков, 7 таблиц, 10 листингов и списка из 5 источников.

Объект исследования — система онлайн-кинотеатра для управления пользователями, фильмами, подписками и историей просмотров.

Цель работы — разработать эффективную и масштабируемую модель данных, поддерживающую основные процессы онлайн-просмотра фильмов в среде СУБД PostgreSQL.

Методология включает проектирование логической и физической схемы базы данных с использованием диаграмм «сущность–связь», реализацию через pgAdmin 14.5 и автоматизированное наполнение данных с помощью скриптов на языке Python.

Ключевые слова: СУБД, PostgreSQL, онлайн-кинотеатр, PostgreSQL.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ 4
1. ПРОЕКТИРОВАНИЕ 5
1.1 Анализ предметной области 5
1.2. Выделение сущностей 6
1.3 Разработка модели бизнес-процессов предметной области 7
1.4 Проектирование инфологической модели базы данных 8
1.5 Проектирование даталогической модели базы данных 9
2. РЕАЛИЗАЦИЯ 13
2.1 Написание скрипта создания базы данных 13
2.2 Заполнение базы данных 15
2.3 Сложные запросы 27
ЗАКЛЮЧЕНИЕ 35
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 36

ВВЕДЕНИЕ

Разрабатываемая система предназначена для автоматизации процесса учета просмотров фильмов пользователями аудиовизуального сервиса. В условиях роста популярности платформ, которые подбирают фильмы, существует необходимость в эффективных инструментах контроля данных о потребителях и их предпочтениях. База данных «Онлайн-кинотеатр» предназначена для удовлетворения перечисленных требований.

Главной целью создания базы данных является систематизация данных о фильмах, их просмотрах и пользователях. «Онлайн-кинотеатр» будет включать в себя такие сущности, как фильм, просмотр, пользователь, подписка, режиссер, устройство и запись.

Актуальность данной базы данных подтверждается несколькими ключевыми преимуществами. Один из них – это автоматизация учета просмотров, которая исключает необходимость ручной обработки данных и возникновения ошибок, связанных с человеческим фактором. Кроме этого, данные хранятся в одном месте, что позволяет эффективнее использовать время работникам платформы при взаимодействии с клиентами для просмотра информации о подписке. Также возможности системы позволят распознавать предпочтения пользователей и персонализировать рекомендации, что увеличит время просмотра фильмов.

1. ПРОЕКТИРОВАНИЕ

1.1 Анализ предметной области

На данный момент уже существуют системы, например, Иви, Okko, Wink и KION, которые предлагают мощные инструменты для управления контентом, пользователями и аналитикой просмотров. Несмотря на их наличие, они зачастую ориентированы на крупные платформы и обладают избыточным функционалом для небольших сервисов или независимых создателей контента. Создание специализированной реляционной базы данных «Онлайн-кинотеатр» позволяет сосредоточиться на ключевых процессах, таких как систематизация фильмов, управление клиентами и аналитика просмотров, уменьшая количество ненужных полей. Это решение позволяет эффективно управлять контентом и улучшать пользовательский опыт без лишних затрат времени и ресурсов.

При проектировании базы данных учитывались такие факторы как хранение в подписке типа подписки, суммы подписки, даты начала и конца подписки, что позволяет отслеживать источники монетизации и управлять доступом пользователей к контенту, также записи о фильмах содержат название, год выпуска, длительность, рейтинг, страну производителя, наличие субтитров и код режиссера для удобного поиска и фильтрации контента по основным критериям. Кроме того, у пользователей зафиксированы логин, пароль, никнейм и код подписки, что обеспечивает безопасное хранение персональной информации и упрощает авторизацию. Запись фиксирует дату записи, что позволяет отслеживать историю просмотра фильмов без избыточных данных.

Для обеспечения безопасности система включает роли администратора, менеджера и клиента. Администратор имеет полный контроль над системой и управляет пользователями, контентом, подписками. Менеджер управляет контентом и может изменять только фильмы. Клиент видит информацию о фильмах, подписке, просмотрах, своих устройствах, имеет возможность редактировать только некоторые поля своих данных.

1.2. Выделение сущностей

Для проектирования базы данных необходимо выделить элементы предметной области. Можно выделить следующие сущности:

1. Пользователь — уникальный идентификатор, логин, пароль, никнейм, код подписки.
2. Подписка — содержит идентификатор подписки, тип подписки, дату начала подписки, дату окончания подписки, сумму платежа.
3. Просмотр — уникальный идентификатор, ссылка на запись, ссылка на устройство, ссылка на пользователя, ссылка на фильм, статус просмотра.
4. Запись — уникальный идентификатор и дата записи.
5. Устройство — содержит идентификатор устройства и модель устройства.
6. Фильм — уникальный идентификатор, название, год, длительность, рейтинг, страна-производитель, наличие субтитров, ссылка на режиссера.
7. Режиссер — уникальный идентификатор, ФИО, дата рождения, жанры.

Главной сущностью является просмотр так как он связывает между собой основные элементы предметной области — пользователей, фильмы, записи и устройства. Вспомогательными сущностями являются пользователь, подписка, запись, режиссер, устройство и фильм.

1.3 Разработка модели бизнес-процессов предметной области

Разработаем диаграмму BPMN (Business Process Model and Notation) для визуализации и описания бизнес-процессов онлайн-кинотеатра. Нотация позволяет наглядно представить последовательность действий, точки принятия решений, потоки данных и зоны ответственности различных участников. Диаграмма охватывает весь процесс, начиная с выбора фильма и заканчивая просмотром. Включены действия, выполняемые пользователем, администратором и онлайн-кинотеатром.

В модели представлены два основных участника:

1. Клиент: Инициирует процесс, выбирает фильм, взаимодействует с онлайн-кинотеатром, начинает просмотр фильма.
2. Онлайн-кинотеатр(Система/Администратор): Центральный участник, отвечающий за прием и обработку запроса на просмотр фильмов. Модель отражает только ключевые точки взаимодействия с онлайн-кинотеатром (получение нового запроса, отправка отчета о результате просмотра – успех или ошибка), без детализации оплаты подписки напрямую на сайте или сторонними сервисами при ее отсутствии.

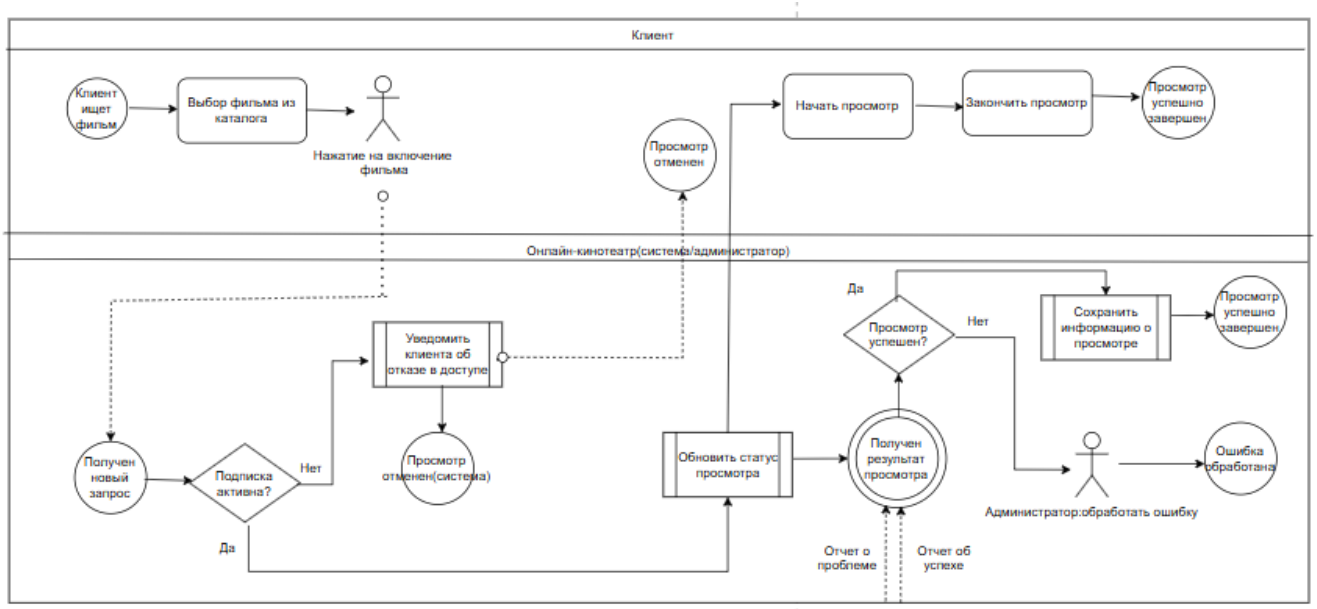


Рисунок 1 – Модель бизнес-процессов предметной области

Основные этапы:

1. Выбор фильма из каталога клиентом

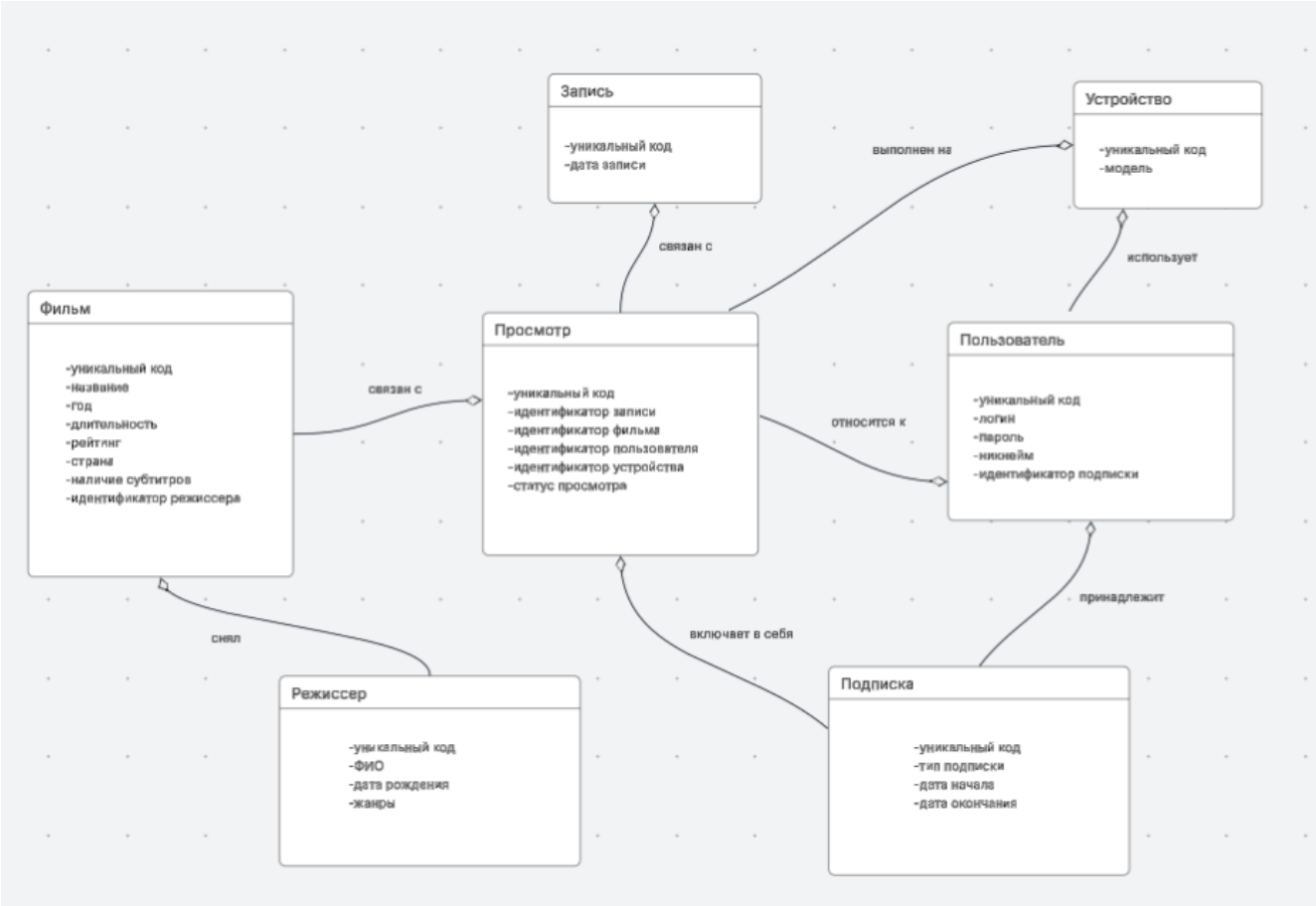
2. Уведомление клиента об отказе в доступе системой при отсутствии активной подписки
3. Обновление статуса просмотра системой
4. Сохранение информации о просмотре системой
5. Обработка ошибки администратором
6. Начало и конец просмотра клиентом

Цель разработки данной BPMN-диаграммы - отразить последовательность действий участников процесса выбора и просмотра фильма в онлайн-кинотеатре, чётко определить зоны ответственности клиента и системы, а также оптимизировать взаимодействие между ними. Это позволяет повысить оперативность обработки запросов на просмотр, обеспечить точность учёта данных о просмотрах, своевременно информировать клиента о статусе доступа и улучшить качество обслуживания за счёт эффективной обработки ошибок администратором.

1.4 Проектирование инфологической модели базы данных

Исходя из сущностей и их свойств, определенных в пункте 1.2, возможно построить инфологическую модель базы данных. Это необходимо для дальнейшего корректного проектирования базы данных:

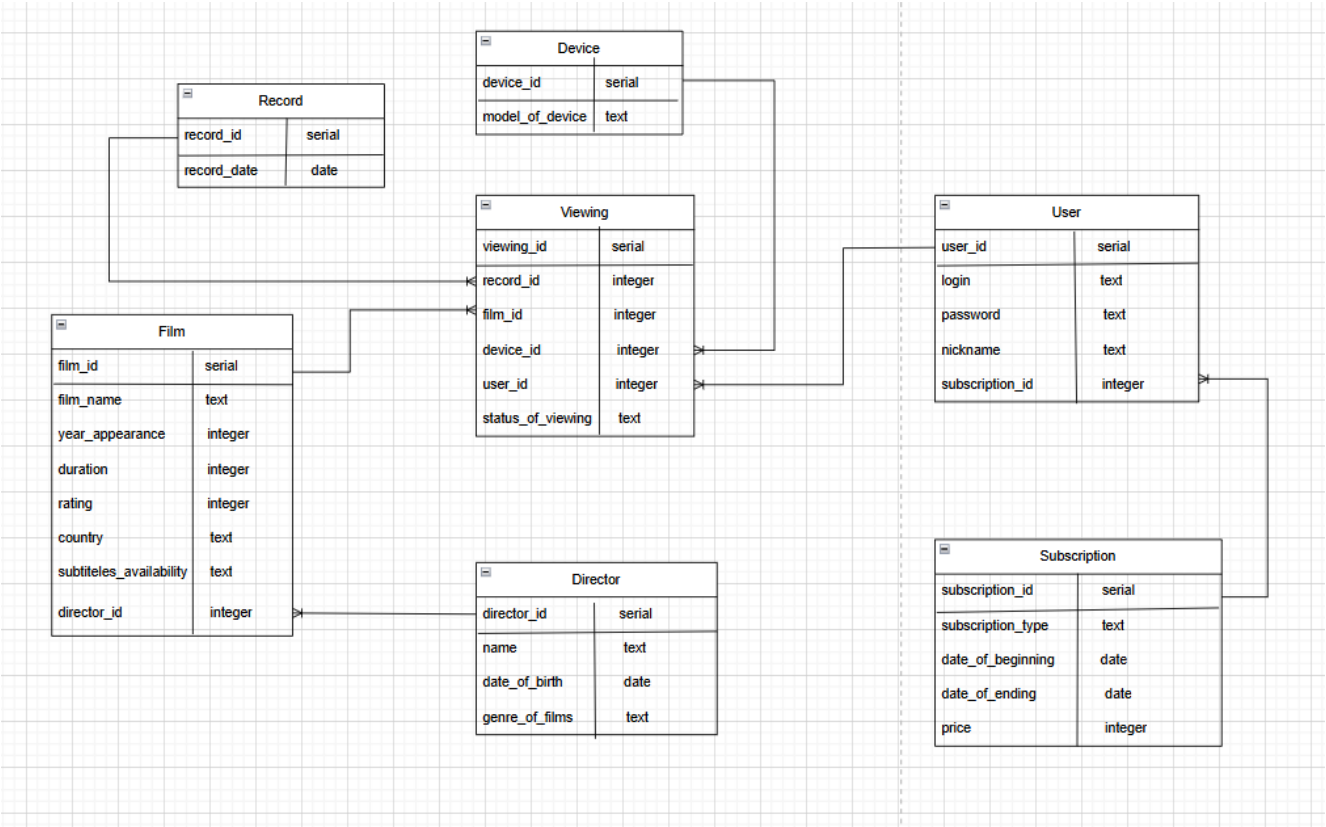
Рисунок 2 – Инфологическая модель базы данных



1.5 Проектирование даталогической модели базы данных

Построим даталогическую модель базы данных, которую можно увидеть на рисунке 3.

Рисунок 3 – Даталогическая модель базы данных



Для того, чтобы пояснить назначение полей в каждой из таблиц, а связи между ними были более очевидными, ниже приведено описание каждой из таблиц и её полей.

Таблица 1 – Описание сущности Record

Название поля	Предназначение поля	Тип данных
record_id	Идентификатор просмотра	целочисленный
record_date	Дата записи	date

Таблица 2 – Описание сущности Device

Название поля	Предназначение поля	Тип данных
device_id	Идентификатор устройства	целочисленный
model_of_device	Модель устройства	строковый

Таблица 3 – Описание сущности Viewing

Название поля	Предназначение поля	Тип данных
viewing_id	Идентификатор просмотра	целочисленный
record_id	Идентификатор записи	целочисленный
film_id	Идентификатор фильма	целочисленный
device_id	Идентификатор устройства	целочисленный
user_id	Идентификатор пользователя	целочисленный
status_of_viewing	Статус просмотра	строковый

Таблица 4 – Описание сущности User

Название поля	Предназначение поля	Тип данных
user_id	Идентификатор пользователя	целочисленный
login	Логин	строковый
password	Пароль	строковый
nickname	Никнейм	строковый
subscription_id	Идентификатор подписки	целочисленный

Таблица 5 – Описание сущности Subscription

Название поля	Предназначение поля	Тип данных
subscription_id	Идентификатор подписки	целочисленный
subscription_type	Тип подписки	строковый
date_of_beginning	Начало подписки	дата
date_of_ending	Конец подписки	дата
price	Цена подписки	целочисленный

Таблица 6 – Описание сущности Film

Название поля	Предназначение поля	Тип данных

film_id	Идентификатор фильма	целочисленный
film_name	Название фильма	строковый
year_appearance	Год выпуска	целочисленный
duration	Длительность в минутах	целочисленный
rating	Рейтинг по 5-бальной шкале	целочисленный
country	Страна происхождения	строковый
subtitles_availability	Наличие русских субтитров	строковый
director_id	Идентификатор режиссера	целочисленный

Таблица 7 – Описание сущности Director

Название поля	Предназначение поля	Тип данных
director_id	Идентификатор режиссера	целочисленный
name	ФИО режиссера	строковый
date_of_birth	Дата рождения	дата
genre_of_films	Жанр фильмов	строковый

2. РЕАЛИЗАЦИЯ

2.1 Написание скрипта создания базы данных

С помощью запросов к базе данных, создадим таблицы, описанные на этапе проектирования. Сами запросы указаны в листинге 1.

Листинг 1 – Скрипт создания таблиц

```
-- Таблица 1: Record (Запись)
CREATE TABLE record (
    record_id SERIAL PRIMARY KEY,
    record_date DATE NOT NULL
);

-- Таблица 2: Device (Устройство)
CREATE TABLE device (
    device_id SERIAL PRIMARY KEY,
    model_of_device TEXT NOT NULL
);

-- Таблица 7: Director (Режиссер)
CREATE TABLE director (
    director_id SERIAL PRIMARY KEY,
    name VARCHAR(70) NOT NULL,
    date_of_birth DATE NOT NULL,
    genre_of_films TEXT NOT NULL
);

-- Таблица 6: Film (Фильм)
CREATE TABLE film (
    film_id SERIAL PRIMARY KEY,
    film_name TEXT NOT NULL,
    year_appearance INTEGER,
    duration INTEGER,
```

```
        rating INTEGER CHECK (rating >= 1 AND rating <= 5),
        country TEXT,
        subtitles_availability TEXT,
        director_id INTEGER REFERENCES director(director_id) ON DELETE CASCADE
    );

    -- Таблица 5: Subscription (Подписка)
    CREATE TABLE subscription (
        subscription_id SERIAL PRIMARY KEY,
        subscription_type TEXT NOT NULL,
        date_of_beginning DATE NOT NULL,
        date_of_ending DATE,
        price INTEGER NOT NULL
    );

    -- Таблица 4: User (Пользователь)
    CREATE TABLE "user" (
        user_id SERIAL PRIMARY KEY,
        login TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        nickname TEXT,
        subscription_id INTEGER REFERENCES subscription(subscription_id)
    );

    -- Таблица 3: Viewing (Просмотр)
    CREATE TABLE viewing (
        viewing_id SERIAL PRIMARY KEY,
        record_id INTEGER REFERENCES record(record_id),
        film_id INTEGER REFERENCES film(film_id),
        device_id INTEGER REFERENCES device(device_id),
```

```
    user_id INTEGER REFERENCES "user"(user_id),
    status_of_viewing TEXT NOT NULL
);
```

2.2 Заполнение базы данных

Для наполнения базы данных был разработан скрипт на Python с использованием библиотек для генерации реалистичных данных.

Сформированные записи согласованы между собой, соответствуют предметной области и типам полей в таблицах, что делает их пригодными для тестирования.

Листинг 2 – Программа на Python для заполнения базы данных значениями

```
import csv
import random
from datetime import timedelta
from dateutil.relativedelta import relativedelta
from faker import Faker
import random
import string
from datetime import datetime

fake = Faker()
fake_ru = Faker('ru_RU')

def random_password(length=10):
    return ''.join(random.choices(string.ascii_letters + string.digits, k=length))

def generate_film_name():
    word_1 = ["Небо", "Равновесие", "Стена", "Голос", "Туман", "Ангел", "Обрыв",
"Секрет",
            "Эхо", "Дождь", "Момент", "Странник", "Ложь", "Обрывок", "Страницы"]
    word_2 = ["алой", "потерянной", "твоей", "волшебной", "чужой", "покинутой",
"алой", "воздушной", "необычной",
            "после", "чужой", "цветной", "воображаемой", "звездной", "хрустальной",
"счастливой", "блестящей"]
    word_3= ["дороги", "тишины", "страсти", "мелодии", "легенды", "доброты",
"реальности", "мечты"]
    return f"{random.choice(word_1)} {random.choice(word_2)}
{random.choice(word_3)}"
```

```
# 1) Director - создаем 100 режиссеров
genres = ["драма", "комедия", "боевик", "фантастика", "ужасы",
        "триллер", "криминал", "мелодрама", "приключения"]

directors = [(i+1, fake_ru.name(),
            fake_ru.date_of_birth(minimum_age=25, maximum_age=80),
            ", ".join(random.sample(genres, k=random.randint(1, 3))))  # 1–3 жанра
           for i in range(100)]
# Сохраняем в CSV
with open('directors.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerow(['director_id', 'name', 'date_of_birth', 'genre_of_films'])
    writer.writerows(directors)

# 2) Film - создаем 1000 фильмов
films = []
for i in range(1000):
    films.append((
        i+1,  # film_id
        generate_film_name(),
        random.randint(1950, 2023),
        random.randint(60, 240),
        random.randint(1, 5),
        fake_ru.country(),
        random.choice(['да', 'нет']),
        random.randint(1, 100)  # director_id
    ))

# Сохраняем в CSV
with open('films.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
```

```
    writer.writerow(['film_id', 'film_name', 'year_appearance', 'duration', 'rating',
'country', 'subtitles_availability', 'director_id'])
    writer.writerows(films)

# 3) Subscription - создаем 1000 подписок
subscription_types = ['базовая', 'стандарт', 'премиум', 'студенческая', 'семейная']
subs_data = []
durations = ['1_month', '6_months', '1_year']

for i in range(1000):
    sub_type = random.choice(subscription_types)
    base_price = {
        'базовая': 500, 'стандарт': 700, 'премиум': 1200, 'студенческая': 450, 'семейная':
900
    }[sub_type]

    start_date = fake.date_between(start_date='-2y', end_date='today')
    duration_type = random.choice(durations)

    if duration_type == '1_month':
        end_date = start_date + relativedelta(months=1)
        final_price = base_price
    elif duration_type == '6_months':
        end_date = start_date + relativedelta(months=6)
        final_price = base_price * 6 - 200
    else:
        end_date = start_date + relativedelta(years=1)
        final_price = base_price * 12 - 500

    subs_data.append((i+1, sub_type, start_date, end_date, final_price))  #
subscription_id
```

```
# Сохраняем в CSV
with open('subscriptions.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerow(['subscription_id', 'subscription_type', 'date_of_beginning',
'date_of_ending', 'price'])
    writer.writerows(subs_data)

# 4) User - создаем 10000 пользователей
fake.seed_instance(0)  # Чтобы генерация была воспроизводимой
unique_fake = fake.unique  # Для генерации уникальных значений

users = []
for i in range(10000):
    users.append((
        i+1,  # user_id
        unique_fake.user_name(),          # Уникальный login
        random_password(),                # Пароль
        fake_ru.first_name(),             # Никнейм
        random.randint(1, 1000)           # subscription_id (1-1000)
    ))

# Сохраняем в CSV
with open('users.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerow(['user_id', 'login', 'password', 'nickname', 'subscription_id'])
    writer.writerows(users)

# 5) Device - создаем 100 устройств
models = [
    "iPhone", "Samsung Galaxy", "iPad", "Xiaomi", "Huawei",
```

```
    "LG TV", "Android TV Box", "Lenovo Tab", "OnePlus", "Nokia",
    "Sony Xperia", "Realme", "Asus Zenfone", "Motorola", "TCL Smart TV"
]

modifiers = [
    "12", "13", "14", "Pro", "Max", "Mini", "Lite", "Plus", "Ultra", "Edge",
    "Air", "Note", "Fold", "Tab", "Go", "Prime", "Smart", "Advance", "Neo", "Play"
]

# Создаём уникальные названия, комбинируя базовую модель и модификатор
device_names = set()
while len(device_names) < 100:
    name = f"{random.choice(models)} {random.choice(modifiers)}"
    device_names.add(name)

# Преобразуем в список кортежей с ID
devices = [(i+1, name) for i, name in enumerate(device_names)]

# Сохраняем в CSV
with open('devices.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerow(['device_id', 'model_of_device'])
    writer.writerows(devices)

# 6) Record - создаем 10000 записей о просмотрах
records = []
for i in range(10000):
    record_date = fake.date_time_between(start_date='-2y', end_date='now')
    records.append((i+1, record_date.date()))  # record_id

# Сохраняем в CSV
```

```
with open('records.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerow(['record_id', 'record_date'])
    writer.writerows(records)

# 7) Viewing - заполняем информацию о просмотрах
statuses = ['начат', 'закончен']
viewing_data = []

for i in range(1000000):  # Генерация 1 миллиона записей
    viewing_data.append((
        i+1,  # viewing_id
        random.randint(1, 10000),  # record_id
        random.randint(1, 1000),   # film_id
        random.randint(1, 100),    # device_id
        random.randint(1, 10000),  # user_id
        random.choice(statuses)    # status_of_viewing
    ))

# Сохраняем в CSV
with open('viewings.csv', mode='w', newline='', encoding='utf-8') as file:
    writer = csv.writer(file)
    writer.writerow(['viewing_id', 'record_id', 'film_id', 'device_id', 'user_id',
'status_of_viewing'])
    writer.writerows(viewing_data)

print("CSV файлы успешно созданы")
```

Проверим данные в таблицах, выполнив запросы на вывод их содержимого. SQL-запросы и результаты их выполнения представлены на рисунках 4-10.

Query Query History

```
1 SELECT * FROM film;
```

Data Output Messages Notifications

Showing rows: 1 to 1000

	film_id [PK] integer	film_name text	year_appearance integer	duration integer	rating integer	country text	subtitles_availability text	director_id integer
1	1	Обрыв хрустальной страсти	1956	131	5	Гватемала	да	50
2	2	Момент цветной доброты	1999	152	1	КНДР	да	84
3	3	Стена твоей реальности	1961	89	4	Алжир	да	77
4	4	Небо цветной тишины	1976	66	5	Доминиканская Республика	да	19
5	5	Равновесие необычной доброты	2009	67	2	Сирия	да	74
6	6	Эхо после мелодии	1982	181	4	Бельгия	да	13
7	7	Секрет покинутой мечты	2006	84	3	Гренада	нет	51
8	8	Страницы цветной реальности	1964	102	4	Самоа	нет	87
9	9	Обрыв покинутой легенды	1983	71	2	Таджикистан	нет	67
10	10	Небо блестящей легенды	2011	227	1	Казахстан	да	78
11	11	Страницы хрустальной мечты	2014	124	3	Суринам	да	81
12	12	Страницы волшебной мечты	1991	141	4	Бенин	нет	32
13	13	Стена цветной страсти	2016	94	3	Эквадор	да	81

Total rows: 1000 Query complete 00:00:00.240

Рисунок 4 – Таблица film

Query Query History

```
1 SELECT * FROM di
```

Data Output Messages

	director_id [PK] integer	name charac
1	1	Нонна
2	2	Пономо
3	3	Федот
4	4	Элеон
5	5	Носко
6	6	Кудря
7	7	Солов
8	8	Тимоф
9	9	Ирина
10	10	Раиса
11	11	Артем
12	12	Анисі
13	13	Черно

Рисунок 5 – Таблица director

Query Query History

```
1 SELECT * FROM device;
```

Data Output Messages Notifications

	device_id [PK] integer	model_of_device text
1	1	Samsung Galaxy Max
2	2	LG TV 13
3	3	Asus Zenfone Tab
4	4	iPhone Prime
5	5	iPhone Advance
6	6	Android TV Box Tab
7	7	Samsung Galaxy Air
8	8	OnePlus 13
9	9	Sony Xperia 13
10	10	Lenovo Tab 12
11	11	Android TV Box Max
12	12	Asus Zenfone Go
13	13	Asus Zenfone Ultra

Total rows: 100 Query complete 00:00:00.147

```
1 SELECT * FROM record;
```

Data Output Messages Notifications

	record_id [PK] integer	record_date date
1	1	2024-05-30
2	2	2024-01-13
3	3	2024-03-03
4	4	2024-10-28
5	5	2025-04-28
6	6	2025-04-09
7	7	2024-04-12
8	8	2025-02-03
9	9	2024-08-03
10	10	2023-05-19
11	11	2024-09-02
12	12	2024-12-16
13	13	2025-03-20

Total rows: 10000 Query complete 00:00:00.168

Рисунок 6 – Таблица record

Query Query History

```
1 SELECT * FROM "user";
```

Data Output Messages Notifications

	user_id [PK] integer	login text	password text	nickname text	subscription_id integer
1	1	achang	rH6mmRi...	Никандр	782
2	2	greenwilliam	y0N2MV4...	Панфил	110
3	3	thull	YfpFX7od...	Стоян	146
4	4	nhoward	NuCH5qpt...	Антонина	991
5	5	juancampos	RWtr71HKlB	Наум	731
6	6	vanessa89	5vkAus7X...	Алексей	199
7	7	kyleblair	cMXoFPY...	Михаил	557
8	8	thomas15	J17CvnP0...	Селиверст	574
9	9	ostewart	G7JTganw...	Валентин	288
10	10	tammywoods	o7OHIAQ6...	Трифон	135
11	11	bryan80	LA8mlI1o...	Демид	3
12	12	cortezraymond	xzkslricR7	Ермолай	218
13	13	salazarmaria	cJYX6kpXvi	Леонид	17

Total rows: 10000 Query complete 00:00:00.167

Рисунок 7 – Таблица device

Query Query History

```
1 SELECT * FROM viewing;
```

Data Output Messages Notifications

	viewing_id [PK] integer	record_id integer	film_id integer	device_id integer	user_id integer	status_of_viewing text
1	1	6534	895	9	84	закончен
2	2	5260	502	2	7903	закончен
3	3	7106	916	13	9010	закончен
4	4	6649	427	44	8145	начат
5	5	4173	880	69	6443	начат
6	6	8826	36	43	5348	закончен
7	7	5731	356	46	3581	начат
8	8	4525	985	64	4921	начат
9	9	9722	304	93	2096	начат
10	10	2209	235	100	6186	начат
11	11	2552	443	34	6501	закончен
12	12	3717	99	30	358	закончен
13	13	6230	486	81	6225	начат

Total rows: 1000000 Query complete 00:00:00.789

Рисунок 8 – Таблица viewing

Рисунок 9 – Таблица user

Query Query History

```
1 SELECT * FROM subscription;
```

Data Output Messages Notifications

	subscription_id [PK] integer	subscription_type text	date_of_beginning date	date_of_ending date	price integer
1	1	стандарт	2024-05-12	2024-06-12	700
2	2	премиум	2023-11-26	2024-05-26	7000
3	3	семейная	2023-12-07	2024-12-07	10300
4	4	базовая	2024-02-03	2024-08-03	2800
5	5	стандарт	2025-03-24	2026-03-24	7900
6	6	базовая	2024-04-19	2024-05-19	500
7	7	стандарт	2023-06-14	2023-12-14	4000
8	8	базовая	2023-08-11	2023-09-11	500
9	9	студенческая	2023-07-31	2024-01-31	2500
10	10	семейная	2023-08-20	2023-09-20	900
11	11	студенческая	2024-11-05	2025-05-05	2500
12	12	семейная	2025-01-24	2026-01-24	10300
13	13	базовая	2024-09-15	2025-03-15	2800

Total rows: 1000 Query complete 00:00:00.164

Рисунок 10 – Таблица subscription

Ниже приведено число записей в каждой таблице.

Таблица 8 – Число записей в каждой таблице

Название таблицы	Число записей
director	100
film	1000
viewing	1000000
record	10000
device	100
user	10000
subscription	1000

2.3 Сложные запросы

Для эффективного мониторинга системы создадим набор SQL-запросов, представленных далее. Подобранные запросы позволяют получать ключевые показатели, значимые для данного типа сервиса.

1. Запрос для получения пользователей, просмотревших больше всего фильмов.

Листинг 3 – Запрос 1

```
SELECT u.nickname, COUNT(v.viewing_id) AS total_views
FROM "user" u
JOIN viewing v ON u.user_id = v.user_id
GROUP BY u.user_id
HAVING COUNT(v.viewing_id) = (
   SELECT MAX(cnt)
   FROM (
      SELECT COUNT(*) AS cnt
      FROM viewing
      GROUP BY user_id
   ) AS sub
);
```

2. Запрос для получения фильмов, которые были просмотрены только в последний месяц.

Листинг 4 – Запрос 2

```
SELECT f.film_name
FROM film f
WHERE f.film_id IN (
   SELECT v.film_id
   FROM viewing v
   JOIN record r ON v.record_id = r.record_id
   WHERE r.record_date >= CURRENT_DATE - INTERVAL '1 month'
);
```

3. Запрос для получения страны с наибольшим количеством просмотров.

Листинг 5 – Запрос 3

```
WITH country_views AS (
    SELECT f.country, COUNT(*) AS view_count
    FROM film f
    JOIN viewing v ON f.film_id = v.film_id
    GROUP BY f.country
)
SELECT country
FROM country_views
WHERE view_count = (SELECT MAX(view_count) FROM country_views);
```

4. Запрос для получения фильмов, не просмотренных ни одним пользователем.

Листинг 6 – Запрос 4

```
SELECT film_name
FROM film
WHERE film_id NOT IN (
    SELECT DISTINCT film_id
    FROM viewing
);
```

5. Запрос для нахождения пользователей с подпиской, истекающей в этом месяце.

Листинг 7 – Запрос 5

```
SELECT u.nickname, s.subscription_type, s.date_of_beginning, s.date_of_ending
FROM "user" u
JOIN subscription s ON u.subscription_id = s.subscription_id
WHERE EXTRACT(MONTH FROM s.date_of_ending) = EXTRACT(MONTH
FROM CURRENT_DATE)
```

```
  AND EXTRACT(YEAR FROM s.date_of_ending) = EXTRACT(YEAR FROM
CURRENT_DATE);
```

6. Триггер для проверки того, что возраст режиссера выше 18.

Листинг 8 – Запрос 6

```
CREATE OR REPLACE FUNCTION check_director_age()
RETURNS TRIGGER AS $$
BEGIN
   IF NEW.date_of_birth > CURRENT_DATE - INTERVAL '18 years' THEN
      RAISE EXCEPTION 'Режиссер должен быть старше 18 лет';
   END IF;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER director_age_check
BEFORE INSERT OR UPDATE ON director
FOR EACH ROW EXECUTE FUNCTION check_director_age();
```

7. Триггер для вывода уведомления об изменении подписки пользователя.

Листинг 9 – Запрос 7

```
CREATE OR REPLACE FUNCTION log_subscription_update()
RETURNS TRIGGER AS $$
BEGIN
   RAISE NOTICE 'Подписка пользователя "%": изменена с % на %', NEW.login,
OLD.subscription_id, NEW.subscription_id;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER user_subscription_update_trigger
AFTER UPDATE OF subscription_id ON "user"
```

```
FOR EACH ROW
WHEN (OLD.subscription_id IS DISTINCT FROM NEW.subscription_id)
EXECUTE FUNCTION log_subscription_update();
```

8. Запрос для нахождения пользователей, которые не смотрели фильмы больше месяца.

Листинг 10 – Запрос 8

```
SELECT u.user_id, u.login, u.password, u.nickname, u.subscription_id
FROM "user" u
WHERE NOT EXISTS (
   SELECT 1
   FROM viewing v
   JOIN record r ON v.record_id = r.record_id
   WHERE v.user_id = u.user_id AND r.record_date >= CURRENT_DATE -
INTERVAL '1 month'
);
```

Выполним запросы и представим их на рисунках 11-18. Получена информация о пользователях, просмотревших больше всего фильмов на рисунке 11.

Query Query History

```sql
SELECT u.nickname, COUNT(v.viewing_id) AS total_views
FROM "user" u
JOIN viewing v ON u.user_id = v.user_id
GROUP BY u.user_id
HAVING COUNT(v.viewing_id) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(*) AS cnt
        FROM viewing
        GROUP BY user_id
    ) AS sub
);
```

Data Output Messages Notifications

	nickname text	total_views bigint
1	Иларион	135
2	Силантий	135

Рисунок 11 – Запрос 1

Получены фильмы, которые были просмотрены только в последний месяц на рисунке 12.

Query Query History

```sql
SELECT f.film_name
FROM film f
WHERE f.film_id IN (
    SELECT v.film_id
    FROM viewing v
    JOIN record r ON v.record_id = r.record_id
    WHERE r.record_date >= CURRENT_DATE - INTERVAL '1 month'
);
```

Data Output Messages Notifications

	film_name text
1	Обрыв хрустальной страсти
2	Момент цветной доброты
3	Стена твоей реальности
4	Небо цветной тишины
5	Равновесие необычной доброты
6	Эхо после мелодии
7	Секрет покинутой мечты
8	Страницы цветной реальности
9	Обрыв покинутой легенды

Total rows: 1000 Query complete 00:00:00.137

Рисунок 12 – Запрос 2

Получена информация о стране с наибольшим количеством просмотров на рисунке 13.

Query Query History

```
1  WITH country_views AS (
2      SELECT f.country, COUNT(*) AS view_count
3      FROM film f
4      JOIN viewing v ON f.film_id = v.film_id
5      GROUP BY f.country
6  )
7  SELECT country
8  FROM country_views
9  WHERE view_count = (SELECT MAX(view_count) FROM country_views);
10
```

Data Output Messages Notifications

	country text
1	Никарагуа

Рисунок 13 – Запрос 3

Получены фильмы, не просмотренные ни одним пользователем на рисунке 14.

Query Query History

```
1  SELECT film_name
2  FROM film
3  WHERE film_id NOT IN (
4      SELECT DISTINCT film_id
5      FROM viewing
6  );
7
```

Data Output Messages Notifications

film_name text

Рисунок 14 – Запрос 4

Получены пользователи с подпиской, истекающей в этом месяце на рисунке 15.

Query Query History

```
SELECT u.nickname, s.subscription_type, s.date_of_beginning, s.date_of_ending
FROM "user" u
JOIN subscription s ON u.subscription_id = s.subscription_id
WHERE EXTRACT(MONTH FROM s.date_of_ending) = EXTRACT(MONTH FROM CURRENT_DATE)
  AND EXTRACT(YEAR FROM s.date_of_ending) = EXTRACT(YEAR FROM CURRENT_DATE);
```

Data Output Messages Notifications

	nickname text	subscription_type text	date_of_beginning date	date_of_ending date
1	Селиверст	премиум	2024-11-13	2025-05-13
2	Серафим	стандарт	2024-05-02	2025-05-02
3	Сила	семейная	2024-05-04	2025-05-04
4	Эрнест	премиум	2024-11-13	2025-05-13
5	Андрей	семейная	2024-05-14	2025-05-14
6	Аскольд	премиум	2024-11-13	2025-05-13
7	Агафья	семейная	2025-04-15	2025-05-15
8	Полина	премиум	2025-04-11	2025-05-11
9	Тимофей	семейная	2024-11-24	2025-05-24

Total rows: 593 Query complete 00:00:00.108

Рисунок 15 – Запрос 5

При добавлении режиссера младше 18 лет выводится ошибка на рисунке 16.

Query Query History

```
INSERT INTO director (name, date_of_birth, genre_of_films)
VALUES ('Петров Денис Андреевич', '2020-10-10', 'Драма');
```

Data Output Messages Notifications

```
ERROR:  Режиссер должен быть старше 18 лет
CONTEXT:  функция PL/pgSQL check_director_age(), строка 4, оператор RAISE

ОШИБКА:  Режиссер должен быть старше 18 лет
SQL state: P0001
```

Рисунок 16 – Запрос 6

При изменении подписки пользователя выводится уведомление на рисунке 17.

Query Query History

```
1 UPDATE "user"
2 SET subscription_id = 7
3 WHERE login = 'sallywalker';
```

Data Output Messages Notifications

```
ЗАМЕЧАНИЕ:  Подписка пользователя "sallywalker": изменена с 4 на 7
UPDATE 1

Query returned successfully in 42 msec.
```

Рисунок 17 – Запрос 7

Получена информация о пользователях, которые не смотрели фильмы больше месяца на рисунке 18.

Query Query History

```
1 SELECT u.user_id, u.login, u.password, u.nickname, u.subscription_id
2 FROM "user" u
3 WHERE NOT EXISTS (
4     SELECT 1
5     FROM viewing v
6     JOIN record r ON v.record_id = r.record_id
7     WHERE v.user_id = u.user_id AND r.record_date >= CURRENT_DATE - INTERVAL '1 month'
8 );
9
```

Data Output Messages Notifications

	user_id [PK] integer	login text	password text	nickname text	subscription_id integer
1	7564	rachel91	sF61t1QPfZ	Фадей	735
2	478	phillipsmichael	mPhPstJM...	Аверкий	596
3	9328	simmonsbrian	lGYW3iZyMD	Тимофей	594
4	7150	uherman	QVmhenRink	Иларион	95
5	570	jeremy94	8nmHjtbDP1	Евстигней	472
6	4554	jenniferfitzgerald	444F1piXIa	Исидор	554
7	4232	gary53	6VE1lLSZsX	Станислав	470
8	4956	coxsharon	mRQxRy64...	Святослав	933
9	8981	michael00	KhXWjzDQNj	Милица	582

Рисунок 18 – Запрос 8

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы были реализованы следующие этапы:

- Проведен анализ предметной области с определением ключевых сущностей и процессов;

- Построена диаграмма бизнес-процессов, отражающая взаимодействие между элементами системы;

- Спроектирована структура базы данных: сформированы инфологическая и даталогическая модели;

- Разработан и реализован SQL-скрипт для создания таблиц и триггеров, обеспечивающих целостность и согласованность данных;

- Составлены и реализованы сложные SQL-запросы для получения аналитической информации;

Итогом стала готовая база данных, предназначенная для хранения, обработки и анализа информации об онлайн-кинотеатре.

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. Изучаем SQL. Генерация, выборка и обработка данных, 3-е изд./ Алан Болье; пер. с англ. И.В. Красикова. — Киев. : “Диалектика”, 2021. — 402 с.: ил. — Парал. тит . англ. ISBN 978-617-7987-01-6 (укр.) ISBN 978-1-492-05761-1 (англ.)

2. Основы технологий баз данных: учебное пособие / Б. А. Новиков, Е. А. Горшкова, Н. Г. Графеева; под ред. Е. В. Рогова. — 2-е изд. — М.: ДМК Пресс, 2020. — 582 с. ISBN 978-5-97060-841-8

3. Стоунз Р., Мэтью Н. PostgreSQL. Основы. – Пер. с англ. – СПб: Символ-Плюс, 2002. – 640 с., ил. ISBN 5-93286-043-X

4. Иванова Г.С., Ничушкина Т.Н. Оформление текстовых документов. Методические указания по оформлению расчетно-пояснительных записок дипломных и квалификационных работ. -М.:Издательство МГТУ им. Н.Э. Баумана, 2004. -10с.

5. PostgreSQL. Основы языка SQL: учеб. пособие / Е. П. Моргунов; под ред. Е. В. Рогова, П. В. Лузанова. — СПб.: БХВ-Петербург, 2018. — 336 с.: ил. ISBN 978-5-9775-4022-3